

CONTENTS

Memorias ROM

Guía de estudio con tolerancia cero — Módulo 3, Arquitectura de Computadoras (FING UdelaR)

0. Por qué existe este problema/tema

1. Qué es una ROM: capacidad y notación

2. Cómo se direcciona una ROM: líneas de dirección y de datos

3. Combinación de ROMs más chicas para construir una más grande

3.a) Mayor tamaño de palabra (más bits de datos, misma cantidad de palabras)

3.b) Mayor cantidad de palabras (más líneas de dirección, mismo ancho de palabra)

3.c) Combinación general: distinta cantidad Y distinto ancho de palabra a la vez

4. ROM como “traductor” o generador de funciones arbitrarias

Ejemplo completo 1: BCD a 7 segmentos

Ejemplo completo 2: función booleana arbitraria de 3 variables

5. Diferencia con RAM (breve — se profundiza en el módulo de memoria)

Ejercicios de repaso rápido (con respuesta)

Resumen ultra-rápido (2 minutos)

Memorias ROM

GUÍA DE ESTUDIO CON TOLERANCIA CERO — MÓDULO 3, ARQUITECTURA DE COMPUTADORAS (FING UDELAR)

0. Por qué existe este problema/tema

Una memoria ROM (Read-Only Memory) es, en el fondo, el circuito combinatorio más “bruto” que existe: en vez de simplificar una función lógica con álgebra de Boole o mapas de Karnaugh (Módulo 2), la ROM **guarda la tabla de verdad completa** en una tabla de consulta (look-up table). Cada combinación posible de entradas es una “dirección”, y en esa dirección está escrito, de antemano, el valor de salida correspondiente.

Esto tiene una consecuencia práctica enorme, y es la razón por la que este módulo se examina tanto: **ningún fabricante hace un chip de ROM del tamaño exacto que vos necesitás**. Los chips vienen en tamaños estándar (1K×8, 2K×4, 32K×16, etc.), y el examen casi siempre te pide *construir* una ROM de un tamaño arbitrario **combinando chips más chicos**. Ahí es donde se pierden puntos: confundir “más palabras” con “más bits por palabra”, errar la cuenta de $\log_2$, o no entender por qué hace falta un decodificador en un caso y no en el otro.

La otra mitad del tema — usar la ROM como “traductor” universal de cualquier función — es conceptualmente más simple, pero exige poder escribir el contenido completo de la memoria fila por fila sin errores de encuadre (¿la fila 5 es la entrada 5 o la salida 5?).

Regla de oro que se repite en TODO el módulo: una ROM de **N palabras × M bits** tiene exactamente **$\log_2(N)$ líneas de dirección** y **M líneas de datos**. Todo lo demás del módulo son variaciones sobre esta única fórmula.

1. Qué es una ROM: capacidad y notación

Definición. Una ROM es una memoria de **solo lectura**: un circuito combinatorio que, dada una dirección de entrada, devuelve siempre el mismo valor de salida (el que fue grabado al fabricarla o programarla). No tiene estado interno ni “recuerda” escrituras — por eso es combinatoria y no secuencial (ver pregunta teórica (c) del Práctico 4).

Notación estándar: una ROM se especifica como

N palabras × M bits (se escribe “N×M” o “NxM”)

donde:

- **N** = cantidad de palabras (posiciones de memoria distintas que se pueden direccionar).
- **M** = cantidad de bits que tiene cada palabra (ancho del dato).

Capacidad total en bits = $N \times M$.

Los prefijos K, M, etc. en este contexto son **potencias de 2**, no de 10: $1K = 2^{10} = 1024$, no 1000. Esto es tolerancia cero: si escribís “8K = 8000” en el examen, la cuenta de líneas de dirección que sigue va a estar mal.

Ejemplo 1.1 — Leer la notación.

Una ROM se especifica como **1K × 8**.

- Cantidad de palabras: $N = 1K = 2^{10} = 1024$.
- Bits por palabra: $M = 8$.
- Capacidad total: $1024 \times 8 = 8192 \text{ bits} = 1 \text{ KB}$ (1 kilobyte).

Verificación: $2^{10} = 1024 \checkmark$ ($1024 \times 8 = 8192$, y $8192/8 = 1024 \checkmark$).

Ejemplo 1.2 — De la capacidad a la notación.

Tengo un chip de ROM de **32K × 16 bits** (aparece en el Ejercicio 5 del Práctico 4).

- $N = 32K = 32 \times 1024 = 32768 = 2^{15}$ palabras.
- $M = 16$ bits por palabra.
- Capacidad total = $32768 \times 16 = 524288 \text{ bits} = 65536 \text{ bytes} = 64 \text{ KB}$.

Verificación: $2^{15} = 32768 \checkmark$. $32768 \times 16 = 524288 = 2^{19} \checkmark$ ($15 + 4 = 19$, porque $16 = 2^4$).

Notación	N (palabras)	N en potencia de 2	M (bits/palabra)	Capacidad total
256×4	256	2^8	4	1024 bits
1K×8	1024	2^{10}	8	8192 bits (1 KB)
8K×8	8192	2^{13}	8	65536 bits (8 KB)
32K×16	32768	2^{15}	16	524288 bits (64 KB)
$2^{18} \times 10$	262144	2^{18}	10	2621440 bits

Error frecuente de tolerancia cero: confundir la capacidad total en bits con la cantidad de palabras. “Una ROM de 8K×8” NO tiene 65536 palabras — tiene 8192 (=8K) palabras de 8 bits cada una. El “8K” de la notación siempre se refiere a N (cantidad de palabras), nunca a la capacidad total.

2. Cómo se direcciona una ROM: líneas de dirección y de datos

Regla:

- **Cantidad de líneas de dirección** = $\log_2(N)$, donde N es la cantidad de palabras. Estas líneas entran al chip (son las entradas $A_0, A_1, \dots, A_{k-1}$).
- **Cantidad de líneas de datos** = M, la cantidad de bits por palabra. Estas líneas salen del chip (son las salidas $D_0, \dots, D_{M-1}$, a veces llamadas $Y_0 \dots Y_{M-1}$).

Además, **todas las ROMs construidas en este curso deben tener dos entradas de control:**

- **CS (Chip Select):** habilita o deshabilita el chip completo. Si CS está desactivado, el chip “no existe” para el resto del circuito.
- **OE (Output Enable):** controla si el chip efectivamente pone el dato en el bus de salida, o si sus salidas quedan en **alta impedancia (tercer estado / tri-state)** — es decir, desconectadas eléctricamente del bus, ni en 0 ni en 1.

El **tercer estado** es lo que permite que varios chips compartan el mismo bus de datos: en cualquier instante, a lo sumo un chip tiene CS y OE activos a la vez (los demás quedan en alta impedancia), así que nunca hay dos chips “empujando” el bus al mismo tiempo con valores distintos.

Ejemplo 2.1 — Contar líneas de una ROM 8K×8.

$N = 8K = 8192 = 2^{13} \rightarrow 13$ líneas de dirección ($A_0 \dots A_{12}$).

$M = 8 \rightarrow 8$ líneas de datos ($D_0 \dots D_7$).

Más CS y OE.

Verificación: $2^{13} = 8192 \checkmark$. Total de pines de A/D+control = $13 + 8 + 2 = 23$ pines (sin contar alimentación).

Ejemplo 2.2 — Contar líneas de una ROM $2^{18} \times 10$ (Ej. 8, Examen Julio 2022).

$N = 2^{18}$ palabras $\rightarrow 18$ líneas de dirección (ya viene dado como potencia de 2, no hace

falta ni sacar logaritmo).

$M = 10$ bits $\rightarrow$ **10 líneas de datos.**

Verificación: $\log_2(2^{18}) = 18 \checkmark$ (identidad directa).

Truco de verificación rápida: armá de memoria la tabla de potencias de 2 hasta 2^{20} (ver tabla de repaso al final). Si el número de palabras no es una potencia de 2 “prolija” (ej. te dan “300 palabras”), redondeá hacia arriba a la potencia de 2 siguiente y aclaralo: vas a necesitar $\text{ceil}(\log_2(N))$ líneas, y quedan direcciones sin usar.

Error frecuente de tolerancia cero: calcular $\log_2$ de la capacidad total ($N \times M$) en vez de $\log_2(N)$. Las líneas de dirección dependen SOLO de la cantidad de palabras, nunca de cuántos bits tiene cada una.

3. Combinación de ROMs más chicas para construir una más grande

Este es el punto que más se toma en parciales y exámenes (aparece en 7 de 9 evaluaciones analizadas). Hay **tres casos**, y la clave de la tolerancia cero es identificar primero cuál (o cuáles) de los dos ejes cambia: el ancho de palabra (M) o la cantidad de palabras (N).

3.A) MAYOR TAMAÑO DE PALABRA (MÁS BITS DE DATOS, MISMA CANTIDAD DE PALABRAS)

Si necesito una ROM con **más bits por palabra** pero la **misma cantidad de palabras** que ya tengo disponible, la solución es poner varias ROMs **en paralelo**:

- Todas las ROMs reciben **las mismas líneas de dirección** (conectadas en paralelo a todas).
- Todas las ROMs reciben el **mismo CS y el mismo OE** (todas están habilitadas o deshabilitadas juntas, siempre a la vez).
- Las salidas de datos de cada ROM se **concatenan** (no se combinan con compuertas): la ROM #1 aporta los bits menos significativos de la palabra de salida, la ROM #2 los siguientes, etc.

Cantidad de ROMs necesarias = $M_{\text{necesario}} / M_{\text{disponible}}$ (bits que necesito, sobre bits que da cada chip).

Ejemplo 3.1 — Práctico 4, Ej.1.b: de 8K×8 a 16K×16 (parte del ancho).

Chip disponible: 8K×8. Necesito 16K×16.

Ancho necesario / ancho disponible = $16/8 = 2$ **ROMs en paralelo** (por cada banco de direcciones — ver 3.b para la otra parte de este ejercicio).

Verificación: cada banco de 2 ROMs de 8 bits en paralelo da $8+8=16$ bits de salida ✓, y ambas comparten las mismas 13 líneas de dirección y el mismo CS/OE ✓.

Error frecuente de tolerancia cero: pensar que para “sumar bits” hay que usar compuertas OR o sumadores sobre las salidas de datos. NO: los bits de dos ROMs en este caso se **concatenan** (van a líneas de salida distintas, $D_0..D_7$ de una y $D_8..D_{15}$ de otra), nunca se combinan lógicamente.

3.B) MAYOR CANTIDAD DE PALABRAS (MÁS LÍNEAS DE DIRECCIÓN, MISMO ANCHO DE PALABRA)

Si necesito **más palabras** pero el **mismo ancho de palabra**, ya no alcanza con paralelo: cada ROM sólo puede direccionar su propio rango de N posiciones, así que hace falta repartir el espacio de direcciones entre varios chips y **decodificar** cuál chip debe responder en cada momento.

Procedimiento:

1. Calculás cuántos chips hacen falta: $N_{\text{necesario}} / N_{\text{disponible}}$.
2. Ese cociente siempre da una potencia de 2 (si los tamaños son “prolijos”); sea 2^k la cantidad de chips.
3. Las **k líneas de dirección más significativas** (las de más peso, las “extra” respecto a las que ya usaba cada chip) se conectan a un **decodificador $k \rightarrow 2^k$** , cuyas salidas van una a una a la línea **CS** de cada chip (una y solo una ROM tiene CS activo en cada instante).
4. Las líneas de dirección **menos significativas** (las que ya traía cada chip) van **en paralelo a todos los chips** (todas reciben la misma dirección “interna”).
5. El **OE** se deja siempre activo en todos los chips (o común a todos): como CS ya garantiza que solo un chip “existe” en cada momento, sus salidas se pueden unir directamente al mismo bus de datos — el resto queda en alta impedancia (tercer estado) y no hay conflicto.

Ejemplo 3.2 — Práctico 4, Ej.1.a: de 8K×8 a 32K×8 (solo cambia N).

Chip disponible: 8K×8 (13 líneas de dirección, 8 de datos). Necesito 32K×8 (mismo ancho, más palabras).

Cantidad de chips = $32K / 8K = 32768/8192 = 4$ **chips**.

$\log_2(4) = 2 \rightarrow$ hacen falta **2 líneas de dirección extra** (las 2 más significativas de las 15 totales que tiene la ROM final: $15 - 13 = 2 \checkmark$) que van a un decodificador $2 \rightarrow 4$, cuyas 4 salidas habilitan el CS de cada uno de los 4 chips.

Las 13 líneas de dirección menos significativas van en paralelo a los 4 chips. Las salidas $D_0..D_7$ de los 4 chips se conectan directamente al mismo bus (solo una tiene el CS activo por vez).

Verificación: $\log_2(32768) = 15$, $\log_2(8192) = 13$, $15 - 13 = 2$, $2^2 = 4$ chips $\checkmark$.

Error frecuente de tolerancia cero: unir las salidas de datos de las distintas ROMs con compuertas OR ("OR cableado"). Esto es exactamente lo que pregunta la parte (f) del cuestionario teórico del Práctico 4: por los retardos de propagación de las compuertas, durante un breve instante el bus puede mostrar un valor incorrecto o fluctuante mientras las señales conmutan. La forma correcta es usar el decodificador para que **como máximo un chip tenga su salida activa (no en alta impedancia) en cada instante**, y conectar las salidas directamente entre sí (bus compartido con tercer estado), nunca con una compuerta lógica de por medio.

3.C) COMBINACIÓN GENERAL: DISTINTA CANTIDAD Y DISTINTO ANCHO DE PALABRA A LA VEZ

Es la combinación de los dos casos anteriores: se arma una "grilla" de chips.

- En el eje horizontal (ancho): tantas ROMs en paralelo como haga falta para completar los bits de salida (caso a).
- En el eje vertical (cantidad de palabras): tantos "bancos" de esas ROMs en paralelo como haga falta, seleccionados por un decodificador sobre los bits más significativos de dirección (caso b).
- **Cantidad total de chips** = (bancos por dirección) $\times$ (chips en paralelo por ancho).

Ejemplo 3.3 — Caso general completo: tengo ROMs de 256×4 y necesito una de $1K \times 8$.

Paso 1 — eje de cantidad de palabras (N): $1K/256 = 1024/256 = 4 \rightarrow$ hacen falta 4 "bancos". $\log_2(4) = 2$ líneas extra de dirección van a un decodificador $2 \rightarrow 4$ que selecciona el CS de cada banco.

Paso 2 — eje de ancho de palabra (M): $8/4 = 2 \rightarrow$ cada banco necesita 2 ROMs en paralelo (mismas líneas de dirección y CS dentro del banco, salidas concatenadas: una da D_0-D_3 , la otra D_4-D_7).

Paso 3 — total de chips = 4 bancos $\times$ 2 en paralelo = **8 ROMs de 256×4 .**

Verificación aritmética: capacidad total necesaria = $1024 \times 8 = 8192$ bits. Capacidad de 8 chips de $256 \times 4 = 8 \times (256 \times 4) = 8 \times 1024 = 8192$ bits ✓ (coincide exactamente, no sobra ni falta capacidad).

Verificación de líneas: dirección final = $\log_2(1024) = 10$ líneas = 8 líneas internas de cada chip ($\log_2(256)=8$) + 2 líneas del decodificador ($8+2=10$ ✓).

Ejemplo 3.4 — Práctico 4, Ej.1.b completo: de $8K \times 8$ a $16K \times 16$ (caso general).

Eje N: $16K/8K = 2 \rightarrow 2$ bancos, 1 línea extra de dirección ($\log_2(2)=1$) a un decodificador $1 \rightarrow 2$ (o un simple inversor + AND, como se ve en la solución del práctico) que selecciona CS de cada banco.

Eje M: $16/8 = 2 \rightarrow 2$ ROMs en paralelo por banco.

Total = 2 bancos $\times$ 2 en paralelo = **4 ROMs de $8K \times 8$.**

Verificación: capacidad necesaria = $16384 \times 16 = 262144$ bits. Capacidad de 4 chips de $8K \times 8 = 4 \times 8192 \times 8 = 262144$ bits ✓. Líneas de dirección: $\log_2(16384)=14 = 13$ (internas de cada chip) + 1 (decodificador) ✓.

Ejemplo 3.5 — Examen Julio 2022, Ej.8.a: ROM de $2^{18} \times 10$ a partir de chips de $2^{15} \times 20$.

Eje N: $2^{18}/2^{15} = 2^3 = 8 \rightarrow 8$ bancos, 3 líneas extra de dirección a un decodificador $3 \rightarrow 8$.

Eje M: el chip disponible da 20 bits, pero solo necesito 10 $\rightarrow$ **sobran bits**, no hace falta paralelo en ancho: uso solo 10 de las 20 salidas de cada chip (las otras 10 quedan sin conectar, o se podrían aprovechar para otra cosa, pero no se necesitan).

Total = 8 chips (nada más, porque en el eje de ancho no hace falta multiplicar).

Verificación: $2^{18} = 262144$, $2^{15} = 32768$, $262144/32768 = 8 = 2^3$ ✓. Capacidad usada: no es un problema que “sobre” ancho de palabra, a diferencia de si sobrara o faltara cantidad de palabras.

Tabla resumen de los 3 casos:

Caso	Qué cambia	Qué se necesita	Direcciones	Salidas de datos
a) Mayor ancho de palabra	M sube, N igual	ROMs en paralelo	Mismas para todas	Se concatenan
b) Mayor cantidad de palabras	N sube, M igual	Decodificador + CS distinto por chip	MSB → decodificador; LSB → todas en paralelo	Se unen al mismo bus (solo 1 chip activo a la vez, tercer estado en los demás)
c) General	N y M suben a la vez	Combinación de a) y b): “grilla” de bancos × paralelo	Igual que b) para las líneas extra, igual que a) dentro de cada banco	Concatenadas dentro del banco, compartidas entre bancos

Truco de verificación universal: después de resolver cualquier ejercicio de combinación de ROMs, verificá SIEMPRE con la cuenta de capacidad total en bits: (cantidad de chips) × (capacidad de cada chip) debe ser $\geq$ (capacidad necesaria), y si los números son “redondos” (potencias de 2 exactas), debe dar EXACTAMENTE igual, no solo “alcanzar”.

4. ROM como “traductor” o generador de funciones arbitrarias

Idea central: el valor de una función lógica (o de cualquier tabla de traducción) depende únicamente de sus entradas. Si la ROM tiene exactamente n entradas de dirección ($A_0 \dots A_{n-1}$) y las hago corresponder una a una con las n variables de la función, entonces **cada una de las 2^n direcciones representa una combinación de entrada distinta**, y lo que grabo en esa dirección es, directamente, el valor de salida que la función debe devolver para esa combinación.

Procedimiento general:

1. Contar cuántas entradas tiene la función (n) → la ROM necesita **2^n palabras** (una por cada combinación posible).
2. Contar cuántos bits tiene la salida de la función (m) → cada palabra de la ROM debe tener **m bits**.
3. Armar la tabla de verdad completa (las 2^n filas) y volcarla, fila por fila, en el contenido de la ROM: dirección = entrada, contenido = salida.

EJEMPLO COMPLETO 1: BCD A 7 SEGMENTOS

Quiero una ROM que traduzca un dígito BCD (4 bits, valores 0 a 9) al patrón de 7 segmentos que hay que encender para mostrarlo en un display (segmentos a,b,c,d,e,f,g; convención “1” = segmento encendido).

- Entradas: 4 bits (dígito BCD) → $2^4 = 16$ direcciones (10 usadas, 6 no usadas/no importa, porque BCD solo llega hasta 9).
- Salidas: 7 bits (uno por segmento) → **ROM de 16×7** como mínimo (en la práctica se usaría un tamaño estándar como 16×8, sobrando 1 bit).

Contenido completo de la ROM (dirección = dígito BCD, contenido = segmentos “abcdefg”):

Dirección (BCD)	Dígito	a	b	c	d	e	f	g
0000	0	1	1	1	1	1	1	0
0001	1	0	1	1	0	0	0	0
0010	2	1	1	0	1	1	0	1
0011	3	1	1	1	1	0	0	1
0100	4	0	1	1	0	0	1	1
0101	5	1	0	1	1	0	1	1
0110	6	1	0	1	1	1	1	1
0111	7	1	1	1	0	0	0	0
1000	8	1	1	1	1	1	1	1
1001	9	1	1	1	1	0	1	1
1010–1111	(no usado)	X	X	X	X	X	X	X

Verificación: $2^4=16$ direcciones totales ✓, de las cuales 10 tienen contenido definido (dígitos 0-9) y 6 son “no importa” (X) porque BCD nunca genera esas combinaciones. Cada fila tiene exactamente 7 bits de salida ✓.

EJEMPLO COMPLETO 2: FUNCIÓN BOOLEANA ARBITRARIA DE 3 VARIABLES

Sea $F(A,B,C) = (A \text{ AND } B) \text{ OR } C$. Uso una ROM con 3 líneas de dirección ($A_2=A$, $A_1=B$, $A_0=C$) y 1 línea de datos.

- Entradas: $3 \rightarrow 2^3 = 8$ direcciones.
- Salida: 1 bit $\rightarrow$ **ROM de 8×1** .

Contenido completo:

Dirección (A B C)	Decimal	$F = (A \cdot B) + C$
000	0	0
001	1	1
010	2	0
011	3	1
100	4	0
101	5	1
110	6	1
111	7	1

Verificación fila por fila: dirección 110 $\rightarrow A=1, B=1, C=0 \rightarrow (1 \cdot 1) + 0 = 1 + 0 = 1 \checkmark$. Dirección 100 $\rightarrow A=1, B=0, C=0 \rightarrow (1 \cdot 0) + 0 = 0 \checkmark$. Dirección 001 $\rightarrow A=0, B=0, C=1 \rightarrow (0 \cdot 0) + 1 = 1 \checkmark$.

Las 8 filas cubren las $2^3=8$ combinaciones posibles, sin faltar ni repetir ninguna.

Este es el mecanismo exacto que se usa en el Ejercicio 2 del Práctico 4 (traductor de mayúsculas $\leftrightarrow$ minúsculas ASCII, $2^7=128$ direcciones de 7 bits cada una), en el Ejercicio 3 (shift aritmético, $2^{11}=2048$ direcciones: 8 bits de dato + 3 de desplazamiento), en el Ejercicio 6 (conversión IEEE754 media $\rightarrow$ simple precisión) y en el Ejercicio 8 del examen (ALU simulada). En todos los casos la lógica es la misma: **contar entradas $\rightarrow 2^n$ direcciones; contar salidas $\rightarrow$ bits por palabra; volcar la tabla de verdad completa.**

Error frecuente de tolerancia cero: al escribir el programa en C que genera el contenido, olvidarse de recorrer TODAS las 2^n direcciones (por ejemplo, un `for` que va de 0 a 127 en vez de 0 a 255 para $n=8$), o invertir el rol de “índice del for” y “dirección de memoria” (el índice del bucle ES la dirección, no un contador aparte). Mirá el ejemplo real del Ej.2 del práctico:

```
void cargar_rom(char* rom[]){
    for(int i = 0; i < 128; i++){           // i recorre las 2^7 = 128 direcciones
        if(i >= 0x41 && i <= 0x5A){        // mayúscula → pasa a minúscula
            rom[i] = i + ('a'-'A');
        } else if(i >= 0x61 && i <= 0x7A){ // minúscula → pasa a mayúscula
            rom[i] = i - ('a'-'A');
        } else {
            rom[i] = i;                    // el resto, sin cambios
        }
    }
}
```

Nota cómo `i` cumple doble función: es el contador del `for` Y la dirección de la ROM (`rom[i]`). Y, dentro de la lógica, el propio carácter ASCII de entrada. Si en el examen usás una variable distinta para “dirección” y para “valor a evaluar” sin que coincidan, es un error grave.

5. Diferencia con RAM (breve — se profundiza en el módulo de memoria)

	ROM	RAM
Operaciones	Solo lectura (una vez grabada, el contenido no cambia en operación normal)	Lectura y escritura
Persistencia	No volátil (conserva el contenido sin alimentación, según la variante)	Volátil (pierde el contenido al apagar, en las variantes típicas DRAM/SRAM)
Uso típico	Tablas fijas, firmware, traductores de código, microprogramas	Memoria de trabajo, datos que cambian en tiempo de ejecución
Naturaleza del circuito	Combinatorio puro	Requiere elementos de almacenamiento (secuencial en su control, aunque el arreglo de celdas en sí varía según la tecnología)

Dato de examen: de las variantes de ROM vistas en el curso (ROM de máscara, PROM, EPROM, EEPROM), la **única que se ajusta estrictamente a la sigla “ROM”** (Read-Only, sin posibilidad de reescritura) es la **PROM** una vez programada — permite grabar los datos después de fabricado el chip, pero no borrarlos ni reescribirlos. El resto de variantes (EPROM, EEPROM) sí permiten reescritura (por luz UV o eléctricamente), por lo que estrictamente no son “de solo lectura” en el sentido literal, aunque se usen como tal en operación normal.

Ejercicios de repaso rápido (con respuesta)

#	Enunciado	Respuesta
1	¿Cuántas líneas de dirección tiene una ROM de 4K×8?	$\log_2(4096) = \mathbf{12 \text{ líneas}}$
2	¿Cuántas líneas de datos tiene una ROM de 4K×8?	8 líneas (M no depende de N)
3	Tengo ROMs de 1K×8. ¿Cuántas necesito para armar una de 4K×8?	$4K/1K = \mathbf{4 \text{ ROMs}}$ (en el eje de direcciones, con decodificador 2→4)
4	Tengo ROMs de 1K×8. ¿Cuántas necesito para armar una de 1K×32?	$32/8 = \mathbf{4 \text{ ROMs en paralelo}}$ (mismas direcciones, salidas concatenadas)
5	Tengo ROMs de 2K×4. ¿Cuántas necesito para armar una de 8K×16?	Eje N: $8K/2K=4$ bancos; eje M: $16/4=4$ en paralelo; total = $4 \times 4 = \mathbf{16 \text{ ROMs}}$
6	¿Por qué NO se debe unir la salida de varias ROMs con compuertas OR?	Por los retardos de propagación, puede haber un instante con valor incorrecto/fluctuante en el bus; se debe usar CS + tercer estado
7	ROM para traducir un carácter ASCII estándar (7 bits) a otro código de 7 bits. ¿Tamaño mínimo?	$2^7 \times 7 = \mathbf{128 \times 7}$
8	ROM que implementa una función de 5 variables booleanas con una salida. ¿Tamaño?	$2^5 \times 1 = \mathbf{32 \times 1}$
9	¿Qué asegura que solo una ROM “hable” en el bus de datos compartido?	Que solo un chip tenga CS activo en cada instante (y OE activo); los demás quedan en alta impedancia (tercer estado)
10	¿Una ROM implementada según la metodología del curso es combinatoria o secuencial?	Combinatoria : la salida depende únicamente del valor actual de las entradas (la dirección), no hay memoria de estados previos
11	ROM de $2^{10} \times 12$ construida con chips de $2^8 \times 12$. ¿Cuántos chips?	$2^{10}/2^8 = 2^2 = \mathbf{4 \text{ chips}}$ (mismo ancho, solo eje de direcciones)
12	¿Qué se conecta a las líneas de dirección menos significativas cuando se	Van en paralelo a todos los chips (son las direcciones “internas” de cada chip)

#	Enunciado	Respuesta
	combina por cantidad de palabras (caso b)?	

Resumen ultra-rápido (2 minutos)

- ROM = **N palabras × M bits**. Capacidad total = $N \times M$ bits.
- **Líneas de dirección** = $\log_2(N)$. **Líneas de datos** = **M**. (Nunca al revés, nunca mezclar con la capacidad total.)
- **CS** habilita/deshabilita el chip entero; **OE** controla si efectivamente saca el dato al bus (si no, queda en **tercer estado / alta impedancia**).
- **Caso a) más ancho de palabra**: ROMs en **paralelo**, mismas direcciones y mismo CS/OE, salidas **concatenadas**. Cantidad de chips = $M_{\text{necesario}}/M_{\text{disponible}}$.
- **Caso b) más cantidad de palabras**: se necesita un **decodificador** sobre los bits de dirección más significativos para activar el **CS** de un solo chip a la vez; los bits menos significativos van en paralelo a todos; las salidas se unen directamente al bus (nunca con compuertas OR). Cantidad de chips = $N_{\text{necesario}}/N_{\text{disponible}}$.
- **Caso c) general**: se combinan a) y b) — una “grilla” de bancos (eje N) × paralelo (eje M). Total de chips = bancos × paralelo.
- **ROM como traductor**: contar entradas $n \rightarrow 2^n$ direcciones; contar salidas $m \rightarrow m$ bits por palabra; volcar la tabla de verdad completa fila por fila (dirección = entrada, contenido = salida).
- **Verificación obligatoria en todo ejercicio**: recalcular (cantidad de chips) × (capacidad de cada chip) y compararlo con la capacidad final pedida — si los números son potencias de 2 exactas, debe dar igual, no solo alcanzar.
- **Nunca OR cableado entre salidas de datos de distintas ROMs** — siempre CS + tercer estado.
- ROM es de **solo lectura** y (según la variante) no volátil; RAM permite lectura y escritura y típicamente es volátil. Solo la **PROM** (ya programada) se ajusta estrictamente a la sigla ROM.